

Trabalho Prático: *Benchmarking* e Análise Empírica de Algoritmos de Ordenação

Formato: Duplas (Recomenda-se, se possível, a formação de duplas mistas entre alunos oriundos da computação e de outras áreas)

Linguagem de Programação: C

Entrega: Link do repositório no GitHub contendo códigos, dados brutos e o Relatório de Execução (PDF)

1. Objetivo

O objetivo deste trabalho é implementar, instrumentar e comparar empiricamente o desempenho de seis algoritmos clássicos de ordenação: **Bubblesort**, **Insertionsort**, **Selectionsort**, **Shellsort**, **Quicksort** e **Heapsort**. O foco está na coleta de dados de execução (tempo, comparações e trocas) em diferentes cenários de entrada e na interpretação analítica dos resultados.

2. Estrutura Base do Código (Esqueleto)

Para garantir que a medição do tempo e a contagem de operações sejam realizadas de forma padronizada por todas as duplas, utilizem obrigatoriamente o modelo de código abaixo como ponto de partida. Vocês só precisam implementar a lógica interna de cada algoritmo de ordenação, incrementando os contadores da estrutura **Mettricas** sempre que houver uma **comparação de chaves** ou uma **troca de posição (escrita)** no vetor.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Estrutura para contabilizar as métricas do algoritmo
typedef struct {
    unsigned long long comparacoes;
    unsigned long long trocas;
    double tempo_segundos;
} Mettricas;

// ---- ESPAÇO PARA AS IMPLEMENTAÇÕES DOS ALUNOS ----

void insertionSort(int *vetor, int n, Mettricas *m) {
    m->comparacoes = 0;
    m->trocas = 0;

    // EXEMPLO DE IMPLEMENTAÇÃO E INSTRUMENTAÇÃO:
    for (int i = 1; i < n; i++) {
        int chave = vetor[i];
        int j = i - 1;

        // Cada entrada no laço testa a condição (j >= 0 e vetor[j] > chave)
        while (j >= 0) {
            m->comparacoes++; // Conta a comparação feita no 'if' conceitual do while
            if (vetor[j] > chave) {
```

```

        vetor[j + 1] = vetor[j];
        m->trocas++; // Conta o deslocamento/escrita
        j--;
    } else {
        break;
    }
}
// Se saiu do laço porque j < 0, a última comparação falhou e já foi contada.
// Se saiu pelo 'break', também já foi contada.
vetor[j + 1] = chave;
}
}

// ---- FUNÇÕES AUXILIARES DE GERAÇÃO DE DADOS ----
void gerarVetorAleatorio(int *v, int n) {
    for (int i = 0; i < n; i++) v[i] = rand() % 100000;
}

void gerarVetorOrdenado(int *v, int n) {
    for (int i = 0; i < n; i++) v[i] = i;
}

void gerarVetorInverso(int *v, int n) {
    for (int i = 0; i < n; i++) v[i] = n - i;
}

int main() {
    int N = 30000; // Exemplo de tamanho
    int *vetor = (int *)malloc(N * sizeof(int));
    if (vetor == NULL) return 1;

    srand(42); // Semente fixa para garantir que todos testem os mesmos dados

    gerarVetorAleatorio(vetor, N);

    struct timespec inicio, fim;
    Metricas m;

    // Medição de tempo de CPU de alta resolução
    clock_gettime(CLOCK_MONOTONIC, &inicio);
    insertionSort(vetor, N, &m);
    clock_gettime(CLOCK_MONOTONIC, &fim);

    // Cálculo do tempo decorrido em segundos
    m.tempo_segundos = (fim.tv_sec - inicio.tv_sec) + (fim.tv_nsec - inicio.tv_nsec)
/ 1e9;

    printf("Resultados:\nTempo: %f s\nComparacoes: %llu\nTrocas: %llu\n",
        m.tempo_segundos, m.comparacoes, m.trocas);

    free(vetor);
}

```

```
return 0;
```

3. Experimentos e Questões do Relatório

Problema 1: O Custo de Operações Teóricas vs. Tempo Real

- **Algoritmos:** Bubblesort (melhorado para parar quando não houver mais trocas e seguir apenas até a posição da última troca), Insertionsort e Selectionsort.
- **Tamanho do Vetor:** $N = 30.000$ elementos.
- **Cenários de Entrada:** 1. **Aleatório:** Elementos distribuídos uniformemente.
- 2. Melhor Caso: Vetor perfeitamente ordenado em ordem crescente.
- 3. **Pior Caso:** Vetor ordenado de forma estritamente decrescente (inverso).
- **O que fazer:** Rode os três algoritmos nos três cenários e preencha uma tabela com o Tempo Total (s), o número de Comparações e o número de Trocas de cada execução.
- **Questão para o Relatório:** Considerando os seus resultados empíricos, o Selectionsort mantém exatamente o mesmo número de comparações tanto no vetor ordenado quanto no vetor inverso. Já o Insertionsort muda drasticamente entre esses cenários. Qual dos dois foi o mais rápido no vetor aleatório? Explique o porquê dessa diferença de comportamento com base na lógica de cada algoritmo.

Problema 2: O Desafio do Quicksort Clássico

- **Algoritmos:** Heapsort e Quicksort clássico (adote a estratégia simples em que o pivô é sempre o último elemento do subvetor corrente).
- **Tamanho do Vetor:** $N = 5.000$ elementos.
- **Cenário de Entrada:** Vetor já ordenado em ordem crescente.
- **O que fazer:** Para este teste específico, execute cada algoritmo 10 vezes, de forma independente, para obter o tempo médio de execução e calcular o desvio padrão.
- **Aviso Importante (Prevenção de Erros):** Como o Quicksort padrão utiliza recursão, rodar um vetor já ordenado (caso degenerado) pode causar estouro da pilha de execução do sistema (Stack Overflow). Para evitar que seu programa quebre, execute o seguinte comando no terminal (Linux/Mac) antes de rodar o executável: `ulimit -s unlimited` (ou aumente o limite de tamanho da pilha nas configurações do seu ambiente/IDE).
- **Questão para o Relatório:** Plote um gráfico de barras comparando o tempo médio do Heapsort e do Quicksort para este cenário (inclua as barras de erro do desvio padrão). Por que o Quicksort demora tanto para ordenar um vetor já ordenado, enquanto o Heapsort lida com ele rapidamente? Explique o que acontece com as partições do Quicksort nesse cenário.

Problema 3: Dados Quase-Ordenados e o Shellsort

- **Algoritmos:** Insertionsort tradicional versus Shellsort (utilizando a sequência clássica de gaps de Shell: divisões sucessivas por 2, ex: $N/2$, $N/4$, ... 1).
- **Tamanho do Vetor:** $N = 50.000$ elementos.
- **Cenário de Entrada:** Vetor **Quase Ordenado**. Para gerá-lo, crie um vetor perfeitamente ordenado e, em seguida, escolha aleatoriamente apenas **0,5% dos elementos** para trocá-los de lugar com seus vizinhos imediatos.
- **Questão para o Relatório:** Registre os tempos de execução de ambos os algoritmos neste cenário. O Shellsort foi concebido como uma evolução do Insertion para permitir saltos longos. No entanto, em um cenário em que o vetor já está quase totalmente ordenado, esses saltos ajudam ou o Insertionsort sozinho consegue ser mais eficiente? Justifique utilizando seus dados numéricos.

4. Diretrizes de Entrega e Infraestrutura

As duplas devem estruturar o projeto de maneira organizada no GitHub, contendo os seguintes itens:

A. Repositório no GitHub

- Criar **um único repositório público** por dupla.
- Organizar os arquivos nas seguintes pastas:
 - `/src`: Todos os códigos-fonte escritos em C (arquivos `.c`).
 - `/data`: Os dados textuais das tabelas e logs gerados pelo terminal (arquivos `.txt` ou `.csv`).
 - `README.md`: Um arquivo de texto explicativo simples na raiz do repositório indicando os comandos utilizados para compilar o código (ex: `gcc main.c -o programa`) e como executá-lo.

B. Relatório de Execução (PDF)

O documento em PDF deve ser incluído no repositório ou submetido na plataforma da disciplina, estruturado com:

- **Identificação**: Nome dos integrantes da dupla.
- **Ambiente de Teste**: Breve descrição do computador onde os testes foram rodados (modelo do processador, quantidade de memória RAM e Sistema Operacional).
- **Apresentação de Dados**: As tabelas geradas no Problema 1 e o gráfico de barras solicitado no Problema 2, gerados no Excel, Google Sheets, Python, R ou em qualquer ferramenta de sua preferência.
- **Análise das Questões**: Respostas fundamentadas e dissertativas às perguntas dos problemas 1, 2 e 3.